Digital Notice Board System

Complete Project Record for Viva Preparation

Prepared On: March 6, 2026

Project Type: Full-Stack Web Application

Current Final Stack: React + TailwindCSS + Node.js + Express + MySQL + JWT

1. Project Evolution (Beginning to Current)

Initial Version (Legacy Scaffold)

- Simple Express server at root (`server.js`) serving static HTML pages from `public/`.
- Basic route setup for admin login and notices.
- Early backend experiments included Mongoose files in root folders (`models/Notice.js`, `controllers/noticeController.js`).

Current Production-Oriented Version (Active)

- Dedicated split architecture with `client/` (React) and `server/` (Express API).
- Database finalized to MySQL as per SRS and course evaluation requirements.
- Role-based system using JWT with two roles: `admin` and `student`.
- Advanced features completed: search, filtering, pagination, read/unread, important flag, toasts, spinner, responsive UI, and near real-time updates via SSE + polling fallback.

2. Final Tech Stack and Why

Frontend: React 18 | Styling: TailwindCSS | HTTP: Axios | Backend: Node.js + Express | Database: MySQL (mysql2) | Auth: JWT + bcryptjs | Security: helmet + validation

Reasoning: React gives reusable component architecture and fast UI development; Express provides lightweight REST API design; MySQL offers relational consistency for users/notices/read states; JWT is suitable for stateless role-based authorization.

3. High-Level Architecture and Flow

```
[React Client]
  |  (Axios + JWT Authorization header)
  v
[Express API Layer]
  |-- /api/auth/login
  |-- /api/notices (CRUD + filters + read/important)
  |-- /api/notices/stream (SSE for live notice change events)
  v
[MySQL Database]
  |-- users
  |-- notices
  |-- notice_reads

Live Update Path:
Admin action -> DB write -> EventEmitter emit -> SSE pushes event -> clients refresh list
Fallback path: periodic polling every few seconds
```

4. Database Schema (Implemented)

Table	Purpose	Main Fields
users	Stores login accounts and roles	id, username, password_hash, role(admin/student), created_at
notices	Stores all published notices	id, title, description, category, created_by, is_important, created_at
notice_reads	Tracks per-user read state for each notice	id, user_id, notice_id, read_at (unique user_id+notice_id)

Indexes exist for performance on category, title, created_at, important flag, and read-join keys.

5. Backend APIs (Current)

Method	Route	Access	Purpose
POST	/api/auth/login	Public	Authenticate and return JWT + role
GET	/api/notices	Student/Admin	List notices with search/category/readStatus/page/limit
GET	/api/notices/categories	Student/Admin	Fetch distinct categories
GET	/api/notices/:id	Student/Admin	Get notice details and auto-mark read

Method	Route	Access	Purpose
POST	/api/notices	Admin	Create notice
PUT	/api/notices/:id	Admin	Update notice
DELETE	/api/notices/:id	Admin	Delete notice
PATCH	/api/notices/:id/read-status	Student/Admin	Mark read or unread per user
PATCH	/api/notices/:id/important	Admin	Toggle important flag
GET	/api/notices/stream	Student/Admin	Server-Sent Events stream for updates

6. RBAC Matrix

Action	Admin	Student
Login	Yes	Yes
View notice list/details	Yes	Yes
Create notice	Yes	No
Edit notice	Yes	No
Delete notice	Yes	No
Mark important	Yes	No
Mark read/unread	Yes	Yes

7. Frontend Pages and Components

- **LoginPage:** role-based sign-in; routes to dashboard/notices by role.
- **NoticeList:** searchable and filterable list with pagination, read tabs, and live updates.
- **NoticeDetails:** full notice content with read/unread toggle.
- **AdminDashboard:** admin controls for edit/delete/important toggle and quick management.
- **CreateNoticeForm:** admin notice creation with validation and important option.
- **Shared Components:** NoticeCard, ProtectedRoute, ToastProvider, LoadingSpinner.

8. Advanced Features Completed

- Search by title/description.
- Category filtering.
- Server-side pagination with metadata.
- Read vs unread notices per user.
- Important notices with highlighted UI styling.
- Toast notifications for success/error actions.
- Loading spinner and graceful error states.
- Responsive interface for mobile and desktop.
- Real-time updates using SSE; fallback polling when SSE is blocked.
- Tab-isolated session handling via `sessionStorage` (prevents one tab login overriding another tab).

9. Security and Reliability Measures

- `helmet` for secure HTTP headers.
- JWT verification middleware on protected routes.
- `requireAdmin` middleware for admin-only endpoints.
- Input validation through `express-validator`.
- Password hashing with `bcryptjs`.
- Centralized error middleware for clean API responses.
- CORS restricted using configured frontend origin.

10. Mapping With Evaluation Phases

Phase 1

- Client-server architecture setup and repository structure finalized.
- Initial DB design translated into implemented SQL schema.
- Baseline documentation and setup flow available in README.

Phase 2

- REST APIs implemented and integrated with frontend.
- JWT authentication + RBAC integrated end-to-end.
- Full CRUD and state flow implemented using React hooks.
- Validation, security middleware, and API error handling completed.

Phase 3

- UI/UX refinement with responsive Tailwind interface and better visual hierarchy.
- Search, filters, pagination, toasts, loader, important and read/unread logic completed.
- Near real-time synchronization via SSE and fallback polling added.
- Project is ready for deployment and viva demonstration workflow.

11. File-Level Final Structure (Active)

```
client/
  src/
    components/ (NoticeCard, ProtectedRoute, ToastProvider, LoadingSpinner)
    pages/ (LoginPage, NoticeList, NoticeDetails, AdminDashboard, CreateNoticeForm)
    services/ (api.js, auth.js)
    App.js
server/
  config/db.js
  controllers/ (authController.js, noticeController.js)
  middleware/ (authMiddleware.js, requireAdmin.js, validate.js, errorMiddleware.js)
  routes/ (authRoutes.js, noticeRoutes.js)
  sql/schema.sql
  utils/noticeEvents.js
  server.js
```

12. Common Viva Questions (Quick Answers)

1. **Why MySQL instead of MongoDB?** SRS and relational read-state tracking fit better with joins and constraints.
2. **How is RBAC enforced?** JWT carries role; backend `requireAdmin` blocks non-admin writes.
3. **How do students and admin coexist simultaneously?** Per-tab session with `sessionStorage`; each tab keeps independent auth state.
4. **How are live updates handled?** SSE push from backend event bus, with fallback polling.
5. **How is read/unread stored?** Separate `notice_reads` table with unique user-notice pair.
6. **How is important notice feature implemented?** `is_important` column + admin-only PATCH route + highlighted UI.
7. **How are inputs validated?** Using `express-validator` on route payloads and params.

8. **How is API secured?** JWT middleware, role middleware, secure headers, hashed passwords, controlled CORS.
9. **How is pagination done?** SQL count + limit/offset + pagination metadata in API response.
10. **What can be future enhancements?** file attachments, notice expiry, email/push notifications, audit logs, unit/integration tests, CI/CD pipelines.

13. Demo Script for Viva (2-3 minutes)

1. Login as `admin`, show Admin Dashboard.
2. Create a notice and mark it important.
3. Edit the notice and save.
4. Open another tab as `student`, show notice appears, mark read/unread.
5. Apply search, category filter, and pagination.
6. Delete notice as admin and show immediate sync in student view.

14. Run Instructions (Final)

1. Run MySQL and execute `server/sql/schema.sql`.
2. In `server/`: `npm install`, configure `.env`, run `npm run dev`.
3. In `client/`: `npm install`, run `npm start`.
4. Open `http://localhost:3000`.

Digital Notice Board Viva Record | Generated from current repository implementation